Python Basics and Programming Concepts

From simple concepts to powerful programs!



1. Introducing Python

- What is Python?
- Why use Python?
- Setting up Python
- Writing your first program

```
print("Hello, Python!")
```

Print statement

```
print("Hello, Python!")
Hello, Python!
```

2. Types and Operations

Variables & data types

Numbers
```
a = 10
b = 3.5
```

Strings
```
name = "Python"
print(name)
```

Lists
```
nums = [1, 2, 3]
nums[0]
```

Tuples
```
t = (1, 2, 3)
t[0]
```

Dictionaries
```
d = {"a":1, "b":2}
d["a"]
```

Files
```
with open("file.txt")
  as f:
    print(f.read())
```

Numeric Types
```
int, float, complex
```

Dynamic Typing
```
x = 5
x = "hello"
```

3. Statements and Syntax

- Assignments
- Expressions
- Statements
- Loops and Iterations
- Comprehensions

```
x = 10
y = x + 5
if x > 5:
    print("Greater")
for i in range(5):
    print(i)
```

If condition / for loop

Simple syntax = powerful ts!

4. Functions

- Function Basics
- Scopes
- Arguments
- Advanced Functions

```
def greet(name):
    return f"Hello, {name}!"

print(greet("Python"))
```

Return Values | Parameters | Default Arguments | *args / **kwargs

Write once, use many times!

5. Modules

- Module Coding Basics
- Module Packages
- Advanced Module Topics

Importing a module

```
import math
print(math.sqrt(16))
```

```
4.0
```

module.py → import module

- os
- sys
- math
- random
- ...

Packages (collection of modules)

Use existing modules or create your own!

6. Classes and OOP

- Class
- Operator Overloading
- Class Designing

Class, object & methods

```
Class Car:
    def __init__(self, brand):
        self.brand = brand
    def show(self):
        return f"Car: {self.brand}"
c = Car("Toyota")
print(c.show())
```

Class → Object → Attributes & Methods

Build real-world models!

7. Exceptions and Tools

- Exception Basics
- Exception Coding Details
- Exception Objects
- Designing With Exceptions
- Parallel System Tools

Try-except error handling

```
try:
    x = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
```

Tools & Libraries

- threading
- multiprocessing
- concurrent.futures
- queue

Handle errors, manage resources, use system tools for better performance!

Write → Run → Build → Grow

Code block = a group of code lines that perform a task.

Python + Logic + Creativity + Endless Possibilities

Python Basics and Programming Concepts

Introducing Python



What is Python?

Python is a high-level, easy-to-learn programming language used for:

- Web Development
- Data Science & AI
- Automation
- Games & More!

Simple. Readable. Powerful.

Why Python?

- Easy Syntax
- Large Community
- Versatile Use
- Cross-Platform



Your First Python Program

Let's print a simple message!

```
print("Hello, Python!")
```



Variables & Data Types

Variables store data. Python supports many data types:

```
name = "Alice"
age = 20
height = 5.5
is_student = True
```

String
Integer
Float
Boolean



Operators

Use operators to perform calculations:

```
a = 10
b = 3
print(a + b)  # 13
print(a * b)  # 30
```

+	-	*	/
Addition	Subtraction	Multiplication	Division



Control Flow

Control the flow of your program:

```
age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

if
else
elif



Loops

Repeat tasks using loops:

```
for i in range(3):
    print("Hello!")
```

Output:
Hello! Hello! Hello!



Conclusion

Python is a beginner-friendly and powerful language. With its simple syntax and wide applications, you can:

- Learn easily
- Build amazing projects
- Grow your career



Start coding with Python today!

Types and Operations – Numbers, Strings

Work with Data in Python!





Numbers in Python

Python supports different numeric types:

int	float	complex
(Integers)	(Decimal)	(Complex)
e.g. 10, -5	e.g. 3.14, -0.5	e.g. 2+3j

Examples:

```
a = 10        # Integer
b = 3.5       # Float
c = 2 + 3j    # Complex
```

Operations on Numbers

You can perform basic arithmetic:

+	-	*	/	%	**
Add	Subtract	Multiply	Divide	Modulus (Remainder)	Power

Examples:

```
a = 10; b = 3
print(a + b)   # 13
print(a - b)   # 7
print(a * b)   # 30
print(a / b)   # 3.333...
print(a % b)   # 1
print(a ** b)  # 1000
```

Use brackets for order of operations! ( )

Strings in Python

Strings are sequences of characters, enclosed in single or double quotes.

Strings can contain letters, numbers and symbols!

Examples:

```
s1 = "Hello"
s2 = 'Python'
s3 = "Learning Python"
```



Operations on Strings

You can use:

+	Concatenation (Join)
*	Repetition
in	Membership Check
len()	Length
[]	Indexing & Slicing

Examples:

```
s = "Hello"
t = "World"
print(s + t)    # HelloWorld
print(s * 3)    # HelloHelloHello
print("H" in s) # True
print(len(s))   # 5
print(s[1:4])   # ell
```

Useful Functions

Python provides helpful built-in functions:

int()	– Convert to integer
float()	– Convert to float
str()	– Convert to string
len()	– Get length
type()	– Check data type

Example:

```
print(type(3.14)) # <class 'float'>
```

Key Takeaways

- Numbers: int, float, complex
- Basic operations: +, -, *, /, %, **
- Strings: text in quotes
- String operations: +, *, in, len(), []
- Use type() to check data type



Numbers and Strings are the building blocks of every Python program!

Lists, Tuples, Dictionaries

Work with Collections in Python!

Let's learn Lists, Tuples and Dictionaries in Python!

These are powerful data structures to store and manage data!

Lists

A collection of items, ordered and changeable.

Key Features:

- Ordered
- Mutable (can be changed)
- Allows duplicates



Example:

```
fruits = ["Apple", "Banana", "Cherry"]
print(fruits[0])  # Apple
fruits.append("Mango")
print(fruits)  # ['Apple', 'Banana', 'Cherry', 'Mango"]
```

Tuples

A collection of items, ordered but unchangeable.

Key Features:

- Ordered
- Immutable (cannot be changed)
- Allows duplicates



Example:

```
numbers = (10, 20, 30)
print(numbers[1])  # 20
```

Tuples are used for fixed data, like coordinates (x, y).

Dictionaries

A collection of key-value pairs.

Key Features:

- Unordered (before Python 3.7)
- Mutable
- Keys must be unique



Example:

```
student = {"name": "Ram", "age": 21, "dept": "Mech"}
print(student["name"])  # Ram
```

Dictionaries are perfect for storing structured data!

Common Operations

Lists	Tuples	Dictionaries
• append()	• count()	• get()
• remove()	• index()	• keys()
• sort()	• len()	• values()
• len()		• items()

More Examples

List

```
nums = [1,2,3]
nums[1] = 5
print(nums)
```

Tuple

```
point = (3, 4)
print(point[0])
```

Dictionary

```
info = {"id": 101}
print(info["id"])
```

Lists are for flexibility, Tuples for safety, and Dictionaries for key-value data!

Key Takeaways

- Lists → ordered & mutable
- Tuples → ordered & immutable
- Dictionaries → key-value pairs
- Choose the right one for your data!

Master these and code smarter!

Files, Numeric Types & Dynamic Typing

Work Smarter with Python!

Files help us store and read data permanently!

Files

File Modes

- "r" – Read
- "w" – Write (overwrite)
- "a" – Append
- "r+" – Read & Write
- "b" – Binary mode

Common Methods

- open() – Open a file
- read() – Read content
- write() – Write data
- close() – Close file
- with open(...) – Safe way

Example:

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Tip: Use "with" to automatically close the file!

Python supports different numeric types for calculations!

Numeric Types

Type	Description	Example
int	Integer numbers	10, -5, 0
float	Decimal numbers	3.14, -0.5
complex	Complex numbers	2 + 3j

Important Points:

- Python automatically converts types.
- Use type() to check the type.
- Supports arithmetic operations (+, -, *, /, %).

Example:

```
a = 10
b = 3.5
print(type(a))  # <class 'int'>
print(type(b))  # <class 'float'>
```

Python is dynamically typed, meaning we don't need to declare variable types!

Dynamic Typing

What is Dynamic Typing?

Variables can hold values of different types during runtime.

Example:

```
x = 10        # int
x = "Hello"   # now str
x = 3.14      # now float
```

Key Features

- No explicit type declaration
- Type changes at runtime
- Simple and flexible

type() Function

```
print(type(x))  # Check type
```

Key Takeaways

- **Files:** Store & read data (using open, read, write).
- **Numeric Types:** int, float, complex.
- **Dynamic Typing:** Python detects types at runtime.
- Write clean, flexible and efficient code!

Keep Practicing... Happy Coding!

Statements and Syntax – Assignments, Expressions, Statements

Write, Compute and Execute in Python!

Python code is written using **statements** and **syntax**!

Syntax is the correct structure of writing Python code!

What are Statements?

Statements are instructions that perform an action.

Examples:

```
print("Hello")   ,   x = 10
```



Assignment Statements

Used to assign a value to a variable.

Example:

```
name = "Ram"
age = 21
pi = 3.14
```

Multiple Assignment:

```
x, y = 10, 20
```

Augmented Assignment:

```
x += 5   # x = x + 5
```

Expressions

Expressions are combinations of values, variables and operators.

Examples:

```
a + b
5 * 3
(a + b) * 2
x > 10
```

Operators:

```
+   -   *   /
%   **   //
==   !=   <   >
<=   >=   and   or
not
```

Statements

Statements are complete lines of code that do something.

Examples:

```
print("Welcome!")
if age >= 18:
    print("Adult")
for i in range(3):
    print(i)
```

Common Statements:

- Input Statement
- Output Statement
- Conditional Statement
- Loop Statement
- Control Statement

Input and Output Statements

Input:
Takes input from user.

Example:

```
name = input("Enter name: ")
```

Output:
Displays result.

Example:

```
print("Hello, Python!")
```

Conditional & Loop Statements

if Statement:
Executes code if condition is true.

Example:

```
if age >= 18:
    print("Adult")
```

Loops:
Repeat code multiple times.

Example:

```
for i in range(5):
    print(i)
```

Syntax Rules

Indentation matters!

- Use correct indentation (in blocks).
- Put strings in quotes "".
- Use parentheses () for expressions.
- End statements with newline.
- Follow Python's case sensitivity.

Key Takeaways

- Statements are actions.
- Expressions produce values.
- Assignment binds values.
- Follow correct syntax.
- Write clean and readable code!

Practice well and write your first Python program!

Loops, Iterations & Comprehensions

Repeat • Iterate • Simplify

Work Smarter with Python!

Loops help us repeat tasks without writing the same code again!

Comprehensions let us create lists, sets and dictionaries in a simple way!

Loops in Python

Loops are used to repeat a block of code.

1. For Loop

Iterates over a sequence (list, range).

Example:

```
for i in range(5):
    print(i)
```

Output:

```
0 1 2 3 4
```

2. While Loop

Repeats while a condition is True.

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Output:

```
1 2 3 4 5
```

Types of Loops

1. **For Loop** – Best for definite iterations.

Example:

```
for i in range(3):
    print("Python")
```

→

```
Python
Python
Python
```

2. **While Loop** – Best for uncertain iterations.

Example:

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

→

```
1
2
3
```

3. **Nested Loops** – Loops inside loops.

Example:

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

→

```
(0, 0) (0, 1) (0, 2)
(1, 0) (1, 1) (1, 2)
```

Comprehensions

A concise way to create lists, sets and dictionaries.

Shorter code, better readability!

List Comprehension

Example:

```
squares = [x*x for x in range(5)]
```

Output: `[0, 1, 4, 9, 16]`

Set Comprehension

Example:

```
unique = {x for x in [1,1,2,3]}
```

Output: `{1, 2, 3}`

Dictionary Comprehension

Example:

```
squares_dict = {x: x*x for x in range(4)}
```

Output: `{0: 0, 1: 1, 2: 4, 3: 9}`

Loop Control Statements

Control the flow of loops.

break	Exit the loop early.
continue	Skip the current iteration.
pass	Do nothing (placeholder).

Example:

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output:

```
0
1
2
```

Nested Loops

Use loops inside another loop.

Nested loops are useful for patterns and matrices!

Example:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j, end=" ")
    print()
```

Output:

```
1 1 1 2 1 3
2 1 2 2 2 3
3 1 3 2 3 3
```

Useful Examples

- Print first 10 natural numbers using a loop.
- Find factorial of a number.
- Print multiplication table.
- Generate even numbers.
- Create a pattern (star, triangle).
- Use list comprehension to square numbers.

Loops + Comprehensions = Powerful Python Programming!

Key Takeaways

- Loops repeat code (`for` and `while`).
- Comprehensions create lists, sets, dicts concisely.
- Use `break`, `continue`, `pass` when needed.
- Nested loops handle complex tasks.
- Write clean, efficient and readable code!

Functions – Function Basics, Scopes, Arguments, Advanced Functions

Write Modular, Reusable and Powerful Code in Python!



What are Functions?

A function is a block of code that performs a specific task.

Example:

```
def greet():
    print("Hello, Python!")
greet()
```

Call a function to execute tits code!

Function Basics

Use def keyword to define a function.

Syntax:

```
def function_name(parameters):
    # code block
    return value
```

Example:

```
def add(a, b):
    return a + b
print(add(5, 3))  # Output: 8
```

Arguments

Pass data to functions using arguments.

Positional Arguments	Keyword Arguments	Default Arguments
add(5, 3)	add(b=3, a=5)	def greet(name="User"): print("Hello", name)

Example:

```
def display(name, msg="Welcome!"):
    print(f"{msg} {name}")
display("Ram")   # Welcome! Ram
```

Scopes

Scope defines where a variable can be accessed.

Local Scope

Variables inside a function.

```
def show():
    x = 10  # local
    print(x)
```

Global Scope

Variables outside functions.

```
x = 20
def show():
    print(x)
```

Use global keyword to modify global variables inside a function.

Advanced Functions

Functions can do much more!

Return Multiple Values

```
return a, b
def get():
    return 1, 2
a, b = get()
```

Lambda Functions

```
square = lambda x:
    x * x
print(square(5))
```

Nested Functions

```
def outer():
    def inner():
        return 5
    return inner()
```

Recursive Functions

```
def fact(n):
    if n == 0:
        return 1
    return n * fact(n-1)
```

Popular Built-in Functions

Python provides many useful built-in functions.

- print() – Display output
- len() – Get length
- type() – Check data type
- input() – Take input
- range() – Generate numbers
- sum() – Sum of numbers

Examples:

```
print(len([1,2,3]))  # 3
print(sum([1,2,3]))  # 6
print(type(10.5))  # float
print(range(1,5))  # 1 2 3 4
```

Key Takeaways

- Functions make code modular and reusable.
- Use def to define and return to get results.
- Understand scopes: local vs global.
- Use arguments for flexibility.
- Explore advanced features like lambda, nested and recursive functions.

Function Flow

Define Function → Pass Arguments → Execute Code → Get Result

Functions = Better Structure + Cleaner Code + Reusability



Modules, Packages & Advanced Module Topics

Organize • Reuse • Extend Your Python Code!

Modules help us organize code into separate files!

Packages group related modules, and advanced topics add more power!

Why Use Them?

- Code Reusability
- Better Organization
- Avoid Name Conflicts
- Build Large Projects

Coding Basics – Modules

A module is a Python file containing functions, classes or variables.

Example: Create a module (math_utils.py)

```
def add(a, b):
    return a + b

def square(x):
    return x * x
```

Save this file and import it in your program!

How to Use?

```
import math_utils                 # import whole module
from math_utils import add        # import specific function
import math_utils as mu           # use alias
```

Example Usage:

```
print(math_utils.add(5, 3))       # Output: 8
print(mu.square(4))               # Output: 16
```

2. Module Packages

A package is a folder that contains multiple modules.



Example:

```
from mypackage
import module1

print(module1.func())
```

Key Points:

- __init__.py makes it a package.
- Organize related modules together.
- Use packages for larger projects.

3. Advanced Module Topics

Standard Library Modules

Python has many built-in modules like:

- math – math functions
- random – random numbers
- os – OS operations
- datetime – date & time

Example:

```
import math
print(math.sqrt(25))
```

Creating Your Own Packages

Structure:

```
mypkg/
├── __init__.py
├── utils.py
└── data.py
```

Use:

```
from mypkg.utils import func
```

External Modules

Install using pip:

```
pip install requests
```

Use:

```
import requests
```

Useful Concepts

- Relative Imports
- Virtual Environments
- Module Search Path (sys.path)

4. More Examples

Using math Module

```
import math
print(math.pi)
print(math.factorial(5))
```

Output:

```
3.141592653589793
120
```

Using random Module

```
import random
print(random.randint(1, 10))
```

Output:

```
7 (random number)
```

Using os Module

```
import os
print(os.getcwd())
```

Output:

```
C:\Users\Python
```

5. Key Takeaways

- Modules = reusable code files.
- Packages = collection of modules.
- Use the standard library.
- Install external modules with pip.
- Organize projects using packages.
- Explore more advanced topics like virtual environments and module search path!

Explore Python Modules and Build Powerful Projects!

Code Smarter, Not Harder!

Classes and OOP

Class, Operator Overloading, Class Designing

Build Powerful Programs with Python!

Classes help us create our own data types and organize code!

OOP lets us use the concepts of Object Oriented Programming!

What are Classes?

A Class is a blueprint for creating objects with data and behavior.

Create objects from the class!

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Creating & Using a Class

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name},
        Age: {self.age}")
```

Usage:

```
s1 = Student("Ram", 20)
c3 = c1may()
```

Output:

```
Name: Ram,
Age: 20
```

Operator Overloading

We can redefine operators like +, == etc. for our class objects.

Example:

```
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag
    def ___add__(self, other):
        return Complex(self.real+other.real,
                       self.imag+other.imag)
```

Usage:

```
c1 = Complex(2, 3)
c2 = Complex(1, 4)
c3 = c1 + c2
print(c3.real,
      c3.imag)
```

Output:

```
3 7
```

Types of Classes

1. Single Class – Basic class
2. Inheritance – Reuse code

Single Class Example:

```
class Car:
    def start(self):
        print("Car Started")
```

Inheritance Example:

```
class ElectricCar(Car):
    def charge(self):
        print("Charging...")
```

Class Designing

Follow good design principles:

- Use meaningful names.
- Keep methods focused.
- Use __init__ for initialization.
- Follow **DRY** principle (Don't Repeat Yourself).
- Use inheritance where needed.

Example – Bank Account

```
class BankAccount:
    def __init_(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        withdraw(self   amount):
    def withdhonk(se lf, amount):
        if amount == self.balance:
            self.balance -=   amount
            return "Withdrawal Successful!"
        else:
            return "Insufficient Balance"
```

More OOP Concepts

Encapsulation	Abstrcation	Polymorphism	Inheritance
Hide data using private members. self,__data	Show essential features only. (using ABC)	Same method, different behavior. (method overriding)	Create new classes from existing ones.

Advanced Examples

Operator Overloading (for ==)

```
def __eq__(self, other):
    return self.name == \
        other.name
```

Compare objects by name!

Class with Multiple Methods

```
class Rectangle:
    def __init.(self, w, h):
        self.w = w; self.h = h
    def area(self):
        return self.w * self.h
    def perimeter(selr):
        return 2*(self.w + self.h)
```

Inheritance with Super()

```
class Dog(Animal);
    def __init_(self, name):
        super()__init__(name)
        self.sound = "Bark"
```

Key Takeaways

- Classes help organize code.
- Objects are instances of classes.
- Operator overloading makes classes more powerful.
- Design classes with good structure.
- Use OOP principles: Encapsulation, Inheritance, Polymorphism.

Practice more... Build Better!

OOP = Power in Python!

Exceptions and Tools – Exception Basics, Exception Coding Details

Handle Errors Gracefully and Build Robust Code in Python!





1. What are Exceptions?

Exceptions are runtime errors that disrupt the normal flow of a program.

Example Errors:

- ZeroDivisionError
- ValueError
- FileNotFoundError
- TypeError

Instead of crashing, we can catch and handle them!

2. Try – Except Block

The basic way to handle exceptions.

```
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except ZeroDivisionError:
    print("Cannot divide by zero!")
except ValueError:
    print("Please enter a valid number!")
```

Output:

```
Enter a number: 0
Cannot divide by zero!
```

3. Exception Coding Details

- Use multiple except blocks for different errors.
- Use else for code when no exception occurs.
- Use finally for cleanup code (e.g., closing files).

Syntax:

```
try:
    # risky code
except ErrorType:
    # handle error
else:
    # runs if no error
finally:
    # always runs
```

4. Common Exceptions

Some frequently used exceptions:

Exception	Cause
ZeroDivisionError	Division by zero
ValueError	Invalid value
TypeError	Wrong data type
IndexError	Index out of range
FileNotFoundError	File not found

You can also create your own custom exceptions!

5. Raising Exceptions

Use raise to throw an exception manually.

```
def check_age(age);
    if age < 18:
        raise ValueError("Age must be 18+")
    return "Welcome!"

check_age(16)   # Raises ValueError
```

6. Exception Handling Tools

assert

Checks conditions during development.

```
assert 1 + 1 == 2,
 "Math Error!"
```

with Statement

Simplifies resource management.

```
with open("file.txt")
  as f:
    data = f.read()
```

Custom Exceptions

Create your own exception class.

```
class MyError(Exception):
    pass
```

7. Best Practices

- Catch only expected exceptions.
- Use meaningful error messages.
- Avoid bare except.
- Clean up resources (files, connections).
- Log errors for debugging.

8. 8. Useful Tools

Debugging Tools

- print()
- debugger (IDE)
- logging module

Modules for Exceptions

- traceback
- sys (for error info)
- logging

Key Takeaways

- Exceptions help to handle errors gracefully.
- Use try, except, else, finally properly.
- Raise exceptions when needed.
- Follow best practices for robust code.



Exception Objects, Designing With Exceptions

Create, Handle and Raise Exceptions Like a Pro!





Exception Objects

Exceptions are classes, so they are objects that can carry information.

Built-in Exception Classes

- **ValueError** – wrong value
- **TypeError** – wrong type
- **KeyError** – missing key
- **IndexError** – invalid index
- **ZeroDivisionError** – divide by zero

Example

```
try:
    x = int("abc")
except ValueError as e:
    print(e)
# Output:
# invalid literal
# for int() with base 10: 'abc'
```

Designing With Exceptions

You can create your own exception classes to handle specific situations.

1. Create a Custom Exception

```
class InsufficientFundsError(Exception):
    pass
```

2. Raise the Exception

```
if balance < amount:
    raise InsufficientFundsError(
        "Not enough balance!")
```

3. Catch and Handle

```
try:
    withdraw(5000)
except InsufficientFundsError
    as e:
    print(e)
```

Example Output

```
Not enough
balance!
```



Exception Object Attributes

Exception objects have useful attributes:

- **args** – arguments passed
- **message** – error message
- **with_traceback()** – add traceback

Example

```
try:
    int("abc")
except ValueError as e:
    print(e.args)   # ('invalid literal...')
```

Chaining Exceptions

Use exception chaining to link related exceptions.

Example

```
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion failed")
    from e
```



Exception Groups (Python 3.11+)

Handle multiple exceptions at once.

Example

```
try:
    ....
except* (ValueError,
    TypeError) as eg:
    print(eg)
```

Best Practices

- Use exceptions for exceptional cases (not normal flow).
- Catch specific exceptions.
- Provide meaningful error messages.
- Keep try blocks small.
- Create custom exceptions for clarity.
- Log errors for debugging.

Handle Errors Gracefully and Write Robust Code!

Key Takeaways:

- Exceptions are objects.
- Create custom exceptions for better control.
- Use try–except wisely and follow best practices.



Parallel System Tools

Run Multiple Tasks Together – Faster & Efficient!

Let's explore Python's tools for parallelism!

Handle multiple tasks at once using threads, processes and more!

- Multithreading
- Multiprocessing
- Concurrent Futures
- Async I/O

1. Multithreading

Run multiple threads in the same process (for I/O-bound tasks).

Example:

```
import threading, time

def task(name):
    print(f"{name} started")
    time.sleep(1) )
    print(f"{name} finished")

thread1 = threading.Thread(target=task,
                           args=("A",))
thread2 = threading.Thread(target=task,
                           args=("B",))
thread1.start(); thread2.start()
thread1.join(); thread2.join()
```

Key Points:

- Great for I/O tasks (files, network).
- Not for CPU-heavy tasks.
- Uses GIL (Global Interpreter Lock).

2. Multiprocessing

Run multiple processes (for CPU-bound tasks).

Example:

```
from multiprocessing import Process

def square(n):
    print(n*n)

p1 = Process(target=square, args=(5,))
p2 = Process(target=square, args=(10,))
p1.start(); p2.start()
p1.join(); p2.join()
```



3. concurrent.futures

High-level API for running tasks using threads or processes.

Example:

```
from concurrent.futures import ThreadPoolExecutor
import time

def work(n):
    time.sleep(1)
    return n*n

with ThreadPoolExecutor() as executor:
    results = list(executor.map(work, [2,3,4]))
    print(results)
```

Features:

- ThreadPoolExecutor (for threads)
- ProcessPoolExecutor (for processes)
- Easy to use for parallel tasks.



4. Asynchronous I/O

Use async and await to handle many I/O tasks efficiently.

Example:

```
import asyncio

async def main():
    print("Start")
    await asyncio.sleep(1)
    print("Done!")

asyncio.run(main())
```

Benefits:

- Handles many I/O-bound tasks (such as web requests).
- Better performance with async.



5. Other Useful Tools

Explore more parallel system tools.



subprocess	threading	multiprocessing	asyncio
Run external programs.	Low-level thread control.	Advanced process control.	Advanced async programming.
subprocess.run()	threading.Thread	multiprocessing.Pool	asyncio.gather()

6. Best Practices

- Choose the right tool: threads for I/O, processes for CPU.
- Use concurrent.futures for simplicity.
- Avoid shared data issues (use locks if needed).
- Use async for many concurrent I/O tasks.
- Test and measure performance.

